

batch × priority 실험 결과 요약 및 분석

Raspberry Pi 5 + Hailo-8L NPU · HailoRT Round-Robin 스케줄러

1. 실험 개요

Detection / Segmentation / Pose 세 모델을 하나의 NPU에서 HailoRT 스케줄러로 동시 추론하며, batch_size와 priority가 추론 성능(지연·처리량·NPU 점유율)에 미치는 영향을 측정했다.

- 변수: batch_size {1, 10, 50, 63} (세 모델 통일), priority {0, 15, 31} (활성 모델별 교차)
- 고정: threshold=1(기본값), timeout=0, 입력 sampled_val2017 673장, INPUT_FPS=0(최대 처리량)
- 설계: 사용모델수(1/2/3) × batch(4) = 252 조건 × 3회 = 756회 (본 보고서 수치는 3회 평균)

2. 핵심 결론

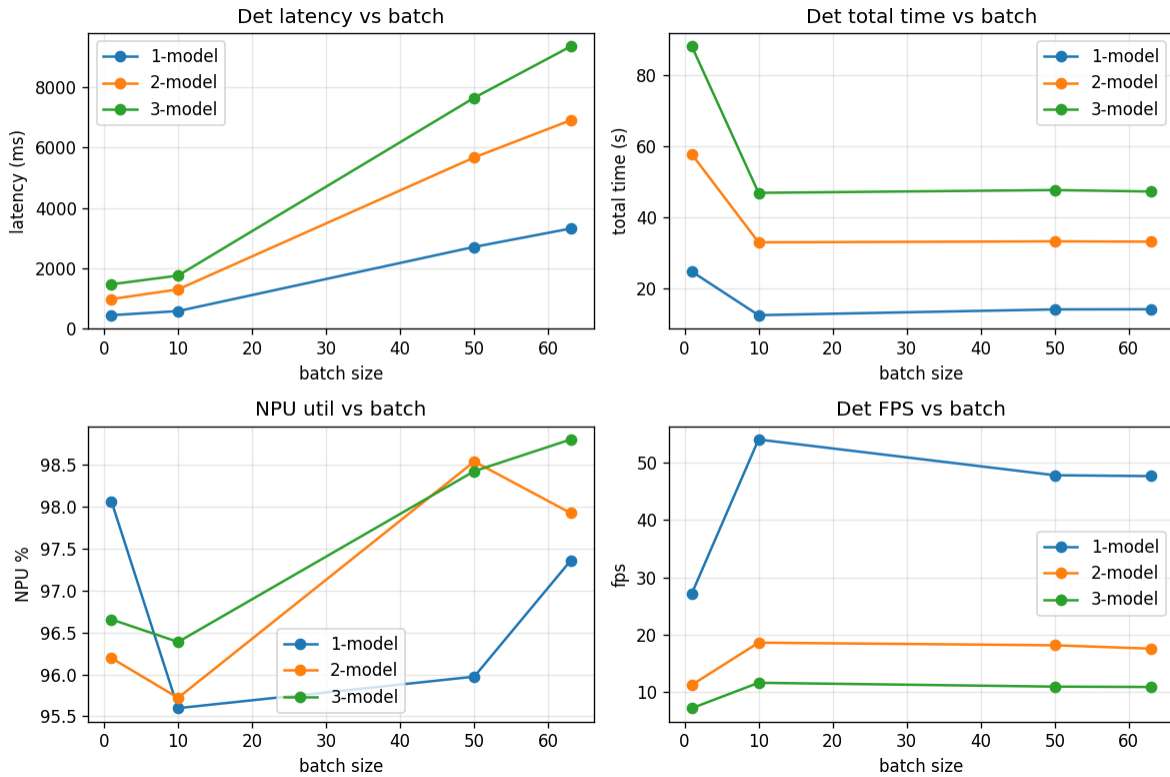
- 우선순위 효과: 우선순위 advantage(자기 우선순위 - 경쟁 모델 최대 우선순위)가 클수록 전체 추론시간 (total_time)이 급감. 상관계수 ≈ -0.78 로 batch 크기와 무관하게 일관.
- 배치 효과: batch가 클수록 프레임당 지연(latency)이 급증(상관 $\approx +0.89$). 그러나 batch=1은 처리량이 낮아 total_time이 오히려 최악 → batch ≥ 10 에서 total_time·FPS가 최적으로 포화되고, 그 이상은 지연만 악화 = 전형적 지연-처리량 트레이드오프.
- NPU 점유율은 전 구간 95~99%로 포화. 우선순위 격차가 크면 낮은 우선순위 모델이 starvation되어 총 처리시간이 크게 늘어남(공정성 이슈).

3. 배치(batch) 효과 — 동일 우선순위 기준

우선순위를 모두 같게 둔(starvation 없는) 조건에서 batch를 키운 결과. 지연은 batch에 비례해 급증하지만, total_time과 FPS는 batch=1에서 나쁘고 batch ≥ 10 에서 포화된다.

model_count	batch	det_lat	seg_lat	pose_lat	tot_det	npu	fps_det
1	1	427.485	481.895	355.125	24.695	98.068	27.257
1	10	563.424	808.767	613.165	12.454	95.598	54.062
1	50	2699.58	3783.427	3175.33	14.078	95.974	47.833
1	63	3309.07	4554.497	3859.66	14.123	97.357	47.68
2	1	963.46	864.802	800.817	57.644	96.196	11.297
2	10	1285.652	1325.643	1241.407	32.934	95.721	18.649
2	50	5676.963	6132.005	5793.002	33.208	98.541	18.192
2	63	6911.49	7559.448	7082.545	33.122	97.927	17.607
3	1	1455.576	1210.458	1197.053	87.983	96.657	7.232
3	10	1745.073	1668.364	1626.604	46.845	96.388	11.655
3	50	7662.708	7784.793	7600.577	47.631	98.426	10.984
3	63	9366.572	9653.939	9397.773	47.219	98.804	10.92

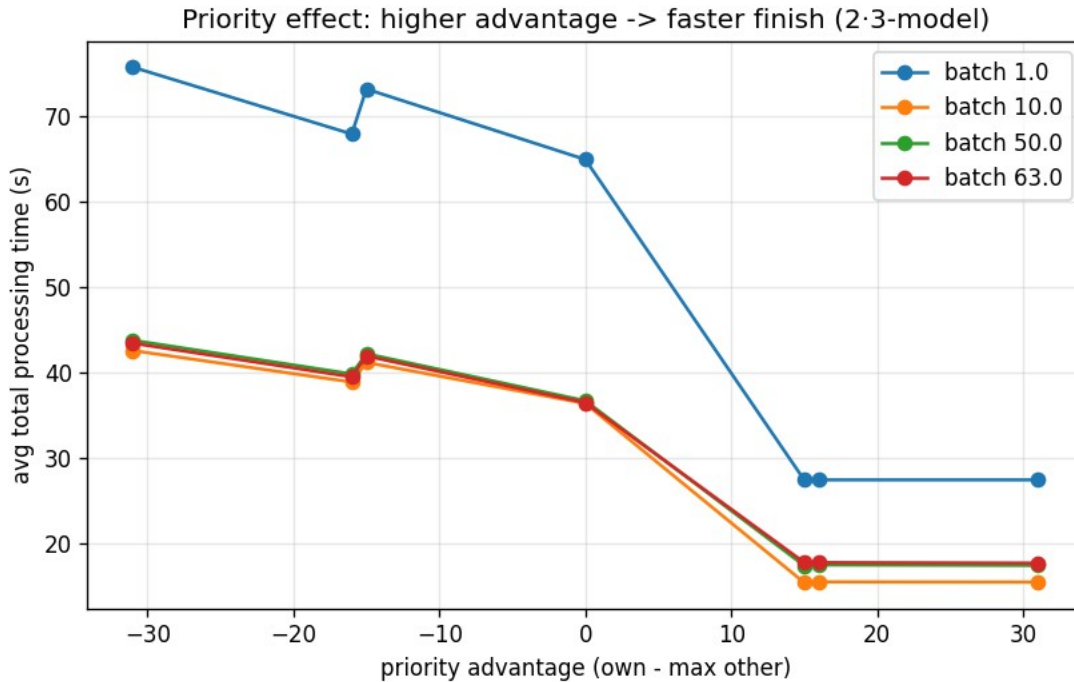
Batch effect at equal priority (no starvation)



해석: batch=1은 파이프라인이 얇아 지연은 낮지만 NPU를 충분히 채우지 못해 처리량(FPS)이 낮고 total_time이 최악이다. batch를 키우면 처리량이 올라 total_time이 급감하지만(≈ 10 에서 포화) 프레임당 지연은 계속 선형 증가한다. 즉 batch는 '지연 vs 처리량'을 맞바꾸는 손잡이이며, 이 환경에선 batch ≈ 10 부근이 균형점이다.

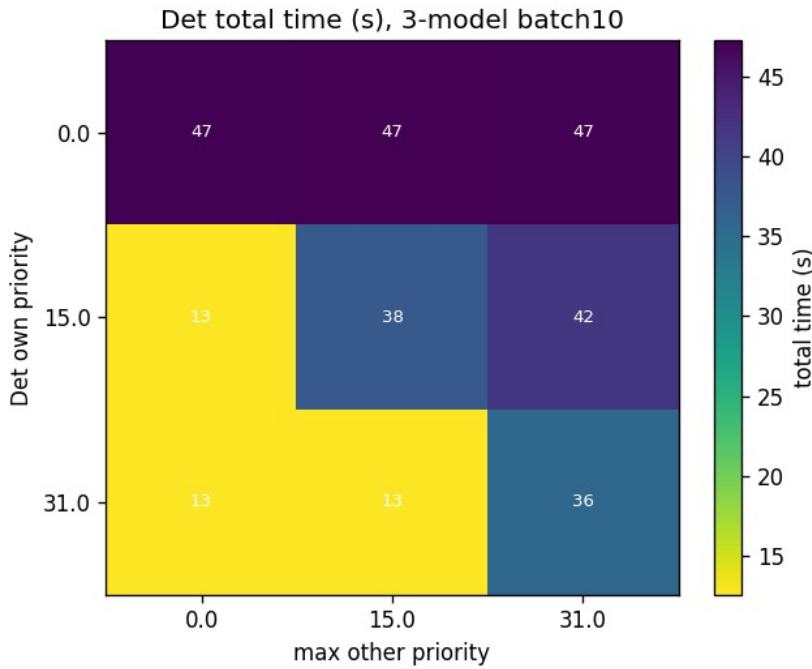
4. 우선순위(priority) 효과

2·3모델 동시 추론에서, 각 모델의 '우선순위 advantage'(자기 - 경쟁 모델 최대)에 따른 전체 추론시간 변화. advantage가 음수(경쟁 모델보다 낮음)면 starvation되어 오래 걸리고, 양수면 우선 처리되어 빨리 끝난다.



advantage가 -31(가장 낮음)일 때 total_time이 최대, +31(가장 높음)일 때 최소로, 모든 batch에서 단조 감소한다. batch=1은 전체적으로 값이 높지만(처리량 낮음) 우선순위에 따른 감소 패턴은 동일하다. 이는 HailoRT 우선순위 스케줄이 실효적으로 작동함을 보여준다.

3모델 batch10에서 Detection의 total_time을 (자기 우선순위 × 경쟁 최대 우선순위)로 본 히트맵:



자기 우선순위가 높고 경쟁 상대 우선순위가 낮을수록 `total_time` 이 작다(우측 하단 방향). 반대로 자기 낮고 상대 높으면 `starvation`으로 최대가 된다.

5. batch ↔ priority 상관관계

두 파라미터와 성능 지표 간 상관계수. 우선순위 `advantage`는 `total_time`·`latency`와 강한 음의 상관, `batch`는 `latency`와 강한 양의 상관을 보인다.

group	pair	corr	n
batch=1.0	advantage vs total_time	-0.735	405
batch=1.0	advantage vs latency	-0.715	405
batch=10.0	advantage vs total_time	-0.783	405
batch=10.0	advantage vs latency	-0.788	405
batch=50.0	advantage vs total_time	-0.786	405
batch=50.0	advantage vs latency	-0.806	405
batch=63.0	advantage vs total_time	-0.782	405
batch=63.0	advantage vs latency	-0.806	405
equal-priority	batch vs det_latency	0.888	72
equal-priority	batch vs total_time_det	-0.416	72
equal-priority	batch vs npu	0.44	108
equal-priority	batch vs fps_det	0.139	72

정리: `priority`는 '누가 먼저/빨리 끝나는가'(공정성·`starvation`)를 지배하고, `batch`는 '지연이나 처리량이나'를 지배한다.

두 손잡이는 서로 독립적으로 작동하되(`priority`-`total_time` 상관이 `batch`에 무관하게 -0.78로 일정), 함께 실시간성·처리량·공정성의 균형점을 결정한다.

6. 유사 실험/연구 사례 비교

다중 모델 동시 추론 스케줄링 연구들이 공통적으로 중점을 둔 항목과, 본 실험과의 연결점.

연구 / 시스템	중점(요약한 것)	우리 실험과의 연결
EdgeServing (Multi-DNN Serving, Edge)	런타임에 모델·exit·batch를 함께 선택해 시스템 전체 SLO 영향 최소화; 시간분할 GPU 공유	batch를 스케줄 knob로 취급 — batch가 지연·처리량을 동시에 좌우함을 정량 확인
Miriam (Real-time Multi-DNN, Edge GPU)	co-running DNN의 지연·처리량 충돌을 elastic kernel로 완화, priority 반영	우리 결과의 지연·처리량 트레이드오프와 동일 문제의식; 우선순위로 완화 관측
Fluid Batching (Edge NPU)	batching(고활용)과 single-sample(저지연)을 preemptive 스케줄로 결합	batch=1 저지연·저처리량 vs 큰 batch 고지연·고처리량 — 같은 트레이드오프의 양 끝
ElasticRoom (Multi-tenant, strong priority)	강한 우선순위로 실시간 저지연+고활용 동시 달성 (Goodput 14~49% ↑)	priority advantage가 클수록 total_time 급감 — 우선순위 실효성 정량화
Fair/Firm RT Scheduling (RL, multi-accel.)	deadline 보상 + 공정성; starvation 방지	우선순위 격차 크면 낮은 모델 starvation(total_time 급증) — 공정성 부재 위험을 데이터로 확인
Heterogeneous co-execution (mobile NPU/DSP)	동시 모델 수 ↑ 시 프로세서별 비균일 성능 저하	1→2→3 모델로 갈수록 지연·total_time 비선형 증가 — 동시성 비용 확인

관련 연구가 공통으로 중점 둔 것: (1) 지연(특히 tail) vs 처리량(goodput) 트레이드오프, (2) 우선순위/시간분할로 실시간성 확보 + starvation·공정성 관리, (3) batch를 정적 상수가 아닌 스케줄 변수로 취급, (4) 가속기 공유 시 동시성에 따른 비균일 성능 저하.

본 실험의 위치: 위 (1)(2)(3)을 batch·priority 2개 파라미터로 실측·정량화했다. 특히 초기 threshold=0 설정에서 발생한 hang은 우선순위 격차에 의한 starvation의 극단 사례로, 관련 연구가 강조하는 anti-starvation 장치(예: SLO-aware priority + debt, fair queuing)의 필요성을 실증한다.

7. 결론 및 시사점

- batch≈10 부근이 이 환경(Hailo-8L, 3모델)의 지연·처리량 균형점. 그 이상 키우면 지연만 나빠진다.
- 우선순위는 실효적으로 작동하나, 격차가 크면 낮은 모델이 굶는다 → 실시간 다중 모델 서비스에선 anti-starvation(최소 보장/타임아웃) 장치가 필요.
- NPU가 상시 포화(95~99%)이므로 threshold/timeout 같은 큐 기반 파라미터는 이 최대 처리량 모드에선 효과가 거의 없다(효과를 보려면 입력 속도 제한 필요).

· 상세 수치·전체 조건은 동봉한 results_batch_priority_SUMMARY.xlsx(요약/케이스별 평균/원본/그래프 시트) 참조.